Machine Learning Recipes – Fundamentals of ML

ELP In-Depth Project 2018-19

Henry Shires

Resources:

- Introduction to Machine Learning: https://towardsdatascience.com/introduction-to-machine-learning-db7c668822c4
- Google Developers ML Recipes: https://www.youtube.com/playlist?list=PLOU2XLYxmsIIuiBfYad6rFYQU_jL2ryal
- Unity ML Agents: https://www.youtube.com/watch?v=32wtJZ3yRfw&index=2&list=PLX2vGYjWbI0R08eWQkO7nQkGiicHAX7IX&t=0s
- Neural Networks Playground: https://playground.tensorflow.org

AI Ethics Principles – Glen Shires:

- "More afraid of humans hacking and doing something bad than ML doing something bad because of their design"
- ML models perform the task they are given to achieve the goal they are given, the model will either correctly or incorrectly perform the task, not deviate from its goal.
- Will ML be used in negative ways to influence and motivate negative human goals?
    - Financial motivation
    - There is a fear of hacking and malicious activity
    - Exposure to ML concepts
    - Always had technology tools for government to do negative events
    - Somebody created the goal, and the computer will follow that goal
    - The bigger risk is that people will use it to do bad things and not the computer attempting to do bad things
    - Being careful of handling important situations with the right people
    - Biggest damages are internet and transferring data
    - Example: said to do something and a computer did it without question
    - Research privacy with systems. Using metadata. Biggest issue today
    - Hacking into systems (i.e., phones, cars, etc.)
- Applications of ML are still being researched and understood
    - Trained based on samples
    - Improving in production and tested, trained offline
    - Cannot train on its own
    - Iterating, constantly improving and gathering new data
    - Solved the problem that you gave it, but in an unrealistic way
    - Completes goals different to how humans would solve it
- Computers do exactly what you tell them to do. Must be specific
- Privacy**:**
    - Digital assistants (Google assistant) always listening for the wake word. Does not send data until after wake word.
    - Processes locally until wake word

Introduction to Machine Learning:

The following examples are programmed in Python, a great data science programming language:

- Prerequisites:
  - Python 3.6 or later compiler and IDE
  - Anaconda
  - Scikit Learn Modules
  - TensorFlow Modules
  - NumPy Modules

Google Developers Recipes:

- **#1 - Hello World (Determining the difference between an Apple and an Orange):**
  - **Classifier:** A function that inputs data and assigns it a label as an output. This uses algorithms such as Supervised Learning. A classifier is like a "box of rules" determined by Training Data and gives an output based on a set of rules.
  - **Supervised Learning:** A technique to write a classifier automatically. It creates it by finding patterns in objects and data inputs
  - Libraries that include this functionality for developers are scikit-learn (uses Python and requires Anaconda)
  - This script performs the basic process for applying a machine learning algorithm to a dataset using Python libraries:
  - The four steps are:
    - 1. Download a dataset (using pandas)
    - 2. Process the numeric data (using NumPy)
    - 3. Train and evaluate learners (using scikit-learn or TensorFlow)
    - 4. Plot and compare results (using matplotlib)

  - **OUR RECIPE (Flow of basic classifier program):**

    Collect Training Data → Train Classifier → Make Predictions
    Input: A description of an object (this instance will be a fruit)
    Output: Predict whether it is an apple or orange based on features

  - **Training Data:** Examples of data for the problems worth solving
    - Writing a function for this instance of collecting training data
    - Training Data is collected by collecting observations, survey data, or measurements called features.
    - Our training data is used via arrays/lists (aggregator variables)
  - **Features:** Descriptions and/or characteristics of a certain label that a program can use to determine the classifier's output
    - Tables are a good way to organize this data
    - The more data collected, the more accurate the results will be

        - Scikit-learn uses real-valued features (integers and floats)
    - **Decision Tree:** A type of classifier very similar to a flowchart in which a condition is given and two options are provided as outputs. The decision tree then states what is ran after a given output. A classifier is called through the scikit learn library:

```
# Defines type of classifier
clf = tree.DecisionTreeClassifier()
```

    After a classifier is instantiated, a learning algorithm is needed for functionality.
    - **Learning Algorithm:** The procedure that creates the rules for a classifier or "empty box of rules". These rules are determined by finding patterns in training data

```
# Defines our Learning Algorithm (fit is for "Finding patterns in Data")
clf = clf.fit(features, labels)
```

    - You can then print data with an input of [160, 0] and output a value of 1, meaning the program recognized the data called as an orange!

- **#2 - Visualizing a Decision Tree (Determine the specific species of an Iris Flower from Iris Dataset):**
    - Types of classifiers include Artificial Neural Networks, Support Vector Machines
    - **Decision Trees (see #1) are easy to use, understand, and are interpretable.** Decision Trees are widely useful since they can be easily understood quickly
    - Goals:
        - Import Dataset
        - Train a Classifier
        - Predict the label for a new flower not in the data set (Important part)
        - Visualize the tree

    - **OUR RECIPE:**

> ***Import dataset → Train Classifier → Make Predictions for new data → Visualize a Decision Tree***
> *Input: A set of data from a list (Iris Flower dataset)*
> *Output: Predict a label for a new flower based on existing data*

    - Metadata: Gives information about the training data. In this case about the features and names of the flowers

```
from sklearn import datasets
import matplotlib.pyplot as plt

iris = datasets.load_iris()
print (iris.feature_names)
print (iris.target_names)
```

    Importing the iris dataset and printing metadata (features and labels from a table)

- Printing the data using a loop can be a quick way to acquire all the data as well
- **Testing Data:** Examples used to "test" the classifier accuracy. This set is NOT part of the training data and simply to test the classifier (Keep these separate)
- "Just like in programming, testing is a very important part of ML"

```
testIdx = [0, 50, 100] # each index where a new set of flowers begin

# training data
trainTarget = np.delete(iris.target, testIdx)
trainData = np.delete(iris.data, testIdx, axis=0)

# testing data
testTarget = iris.target[testIdx]
testData = iris.data[testIdx]
```

- Make sure to import modules! Two sets of variables (training and testing) that remove two items of data and test them
- In this case, the target variables are labels, and the data variables are features of those labels (see #1)
- Next, define a classifier from #1 (train and predict functions. See for code) but predict what the testing data is to confirm that the classifier is properly trained
- **A decision tree is read from top to bottom, with certain conditions being met based on given inputs. Once no more conditions can be met, a decision is made**
- More to Learn:
  - How are trees built automatically from examples?
  - How well do they work in practice?
- *Choosing good features is one of your most important jobs! Good features allow for the accurate generation of a decision tree and great results.*

- **#3 - What Makes a Good Feature? (Difference between Greyhounds and Labradors):**
  - **Binary Classification:** Two different features in a training dataset. The easiest form of features since you only need two proficient ones.
  - Due to the variation of data in the world, consider the difference of values for each type of feature to determine its effectiveness in a program
  - Features capture different types of information, otherwise if/then logic is all that is needed
  - **Thought Experiment: Example:** "If you were the classifier trying to predict the label, what else would you need to do so?" The amount of features you would need to solve the problem is the amount you need in your program
    - If a feature is 50-50, or similar results are in both labels, *don't use it,* as it can hurt the classifier results
    - Independent features are recommended

        - **Independent Features:** Use different types of information (example: height in inches vs height in cm are measuring the same thing)
    - Features should be easy to understand, as simpler relationships are easier to learn
- **In summary:**

> **Ideal Features are:**
> - Informative
> - Independent
> - Simple

- **#4 - Writing a Pipeline:**
    - **Pipeline:** The flow of an ML program, from importing modules, training a classifier, to predicting labels or outputs from data
    - To review, you can use two partitions of data to train and test the classifier:
        - Training Data that includes collected examples of data
        - Testing Data that includes theoretical data that tests the classifier to determine if it has been properly trained
    - Features and labels can be represented by simple function notation:

> Features → **f(x) = y** ← Labels

    - While there are many different types of classifiers, they each have a similar interface
    - A common way to think of it in Python is:

```
def classify(features):
    # do some logic
    return label
```

    - However, an algorithm is preferred to do it for the developer and become trained.
    - The body of the function is written algorithmically or learned. Start with a model instead:
        1. **Model:** The rules that define the body of a function
    - *Using a line as a classifier:*
        1. Graphing a line to represent a classifier, and adding dots on one side or the other (both sides being features)
        2. The parameters of the function determine the classifier
            - Ex: y = mx+b with m and b that change the line
            - Start with one training example and create the classifier from there, then add another and see if the classifier gets it right. If not, try different parameters, but if so, the classifier works.
            - **“Learning”:** Using training data to adjust the parameters of a model

- An example of importing some cool Scikit Learn modules and experimenting with classifiers (see #5 for KNeighborsClassifier):

```
from sklearn import datasets
iris = datasets.load_iris()

x = iris.data # features
y = iris.target # labels

from sklearn.model_selection import train_test_split
# a built in function that allows for automated train/test splitting of the data.
# features and label values are used, and 'test_size' defines how much of the data is split into train or test
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = .5)

from sklearn.neighbors import KNeighborsClassifier # train a new type of classifier
clf = KNeighborsClassifier()

clf.fit(x_train, y_train)
predict = clf.predict(x_test)
print(predict) # this prints the type of iris the classifier predicts for each row of testing data

from sklearn.metrics import accuracy_score
print(accuracy_score(y_test, predict))
```

- **Neural Network (Example: https://playground.tensorflow.org):** A more complex classifier (see Neural Network notes)

- **#5 - Writing Your Own Classifier: Writing a Classifier instead of Importing One**
  - **Steps to Writing Your Own Classifier:**
    1. Comment out imports of any classifier used:
       - Make sure your accuracy rating on your training data is typically above 90% or a float of 0.90 with one or two features (see code from #4)
    2. Implement a Class:

```
1   class ScrappyKNN(): # Scrappy = bare-bones
15      clf = ScrappyKNN() # sets classifier var to class
```

    3. Understand the interface:
       - Add functions based on what functions we needed from the scikit learn module:

```
def fit(self, x_train, y_train):  # takes the features and labels as input
    pass

def predict(self, x_test): # inputs the features from testing data and outputs predictions
    pass
```

    4. Get pipeline working:
       - Make sure to have an accuracy rating averaging 33% for this instance due to 3 types of labels in the dataset

5. How a KNN classifier works (KNearestNeighbor):
    - Given training data on a scatter plot, memorized in the “fit function”, predict the label for a given or random point on the plot
    - It performs this action by finding the training point closest to the testing point and assumes that the testing point has the same label as the training point (hence, *nearest neighbor*)
    - If the data is equidistant, it is determined by using *k*, or the number of neighboring data to consider (hence, *K*NN)
        - *If k is 1, look at the closest training point*
        - *If k is >1, look at k closest training points*
        - The majority data points are chosen as the label for the new point
6. Measure distance (find the nearest neighbor):
    - Uses the Euclidean distance formula [$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$]
    - With more features adds more terms to the formula (dimensions)
7. Import nearest neighbor algorithm:
    - Calculate the distance of all training points
    - Predict the testing data’s closest neighbor and assign it that label
    - The source code that implements Euclidean distance formula and functions for recording the shortest distance:

```
from scipy.spatial import distance

def euc(a, b):
    return distance.euclidean(a, b) # calculate point distance

class ScrappyKNN(): # Our classifier (Scrappy = bare-bones)
    def fit(self, x_train, y_train):  # takes the features and labels as input
        self.x_train = x_train # features
        self.y_train = y_train # labels

    def predict(self, x_test): # inputs the features from testing data and outputs predictions
        predictions = [] # returning list of predictions
        for row in x_test:
            label = self.closest(row) # finds closest training point from test point
            predictions.append(label)
        return predictions

    def closest(self, row): # loops over all of training points and updates to the closest one so far
        bestDist = euc(row, self.x_train[0]) # records best distance from testing point to training point
        bestIndex = 0 # records the best index
        for i in range(len(self.x_train)):
            dist = euc(row, self.x_train[i])
            if dist < bestDist:
                bestDist = dist
                bestIndex = i
        return self.y_train[bestIndex]
```

8. Run pipeline:
    - If accuracy is greater than 90%, KNN classifier is correctly programmed and trained

- Is the KNN algorithm worth it?
    - Pros:
        - Relatively simple

        - Cons:
            - Computationally intensive
            - Hard to represent relationships between features

The next few recipes require [Docker Toolbox] and basic knowledge of bash commands:

- **#6 - Train an Image Classifier with TensorFlow for Poets:**
    - Building a classifier that finds the difference between images
    - TensorFlow for Poets:
        1. High-level code
        2. Powerful classifier
        3. Open source machine learning library that's useful for using Deep Learning
    - Deep Learning: No feature engineering is needed, only the feature sources. The classifier is powerful enough to understand the raw differences of features
        1. When using deep learning, the classifier used is called a Neural Network
            - A neural network can learn more complex functions
            - Input → Hidden Layer → Hidden Layer → Output
    - TF Learn:
        1. High-level ML library on top of TensorFlow
        2. Syntax is similar to scikit-learn
        3. Formerly known as SK Flow
    - MobileNet: A small efficient convolutional neural network.
        1. Convolutional: Means that the same calculations are performed at each location in the image.
    - Instead of training a classifier from scratch, TensorFlow is starting with an existing classifier called Inception:
        1. One of Google's best image classifiers
        2. Open source
        3. Trained on 1.2 million images
        4. Training took 2 weeks
    - Retraining (or Transfer Learning):
        1. Saves a lot of time
        2. Leverages prior work with less training data
    - To train a good classifier, use diverse and large amounts of features
- **#7 - Classifying Handwritten Digits with TF.Learn:**
    - Objective: Classifying handwritten digits from the MNIST dataset and predict a digit given an image:
        - Installation
        - Download dataset
        - Visualize images
        - Train a classifier
        - Evaluate

    - Visualize Weights
        1. Code examples can be found here: http://192.168.99.100:8888/notebooks/ep7.ipynb (If a token is required, make sure to run *docker run -it -p 8888:8888 TensorFlow/tensorflow:1.12.0-rc2* in docker and copy the url with the correct IP address to bypass it)
    - When working with images, no feature engineering is required and raw pixels are used instead. The classifier does the rest (# of pixels = # of features)
    - Edges of the data are stacked, as each feature passes through each node, starting with the input node and moving through edges. Each edge has a weight so the feature is multiplied by the weight of each edge and each output node gathers evidence as to which number the feature is.
    - The more evidence = the increased chances of that being the correct label

$$\text{evidence } y_i = \sum W_{i,j} x_j$$ The sum of the value of the pixel intensities multiplied by the value of the weights.

y = labels, x = features, W = weights

    - Weights adjusted by gradient descent: Setting them properly means more accurate classification
        - Begin with random weights
        - Gradually adjust to better values
    - Accuracy is easily obtained at 99% with Deep Learning
    - Weights can be visualized using color to outline the features and how much evidence is being put to the test

- **#8 - Writing a Decision Tree Classifier from Scratch:**

    - **Source Code (Credit to Google Developers):** https://github.com/random-forests/tutorials/blob/master/decision_tree.ipynb

    - Outline:
        1. Dataset
        2. Preview completed tree
        3. Build the tree
    - Objective: To predict a type of fruit based on features such as color and size
    - Example Training Data:

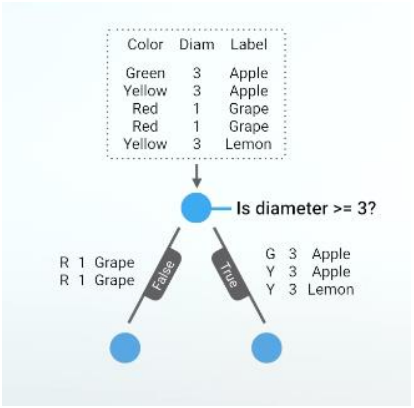
Color	Diameter	Label
Green	3	Apple
Yellow	3	Apple
Red	1	Grape

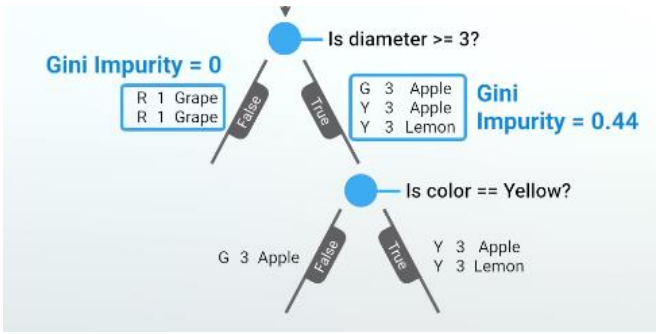
Red	1	Grape
Yellow	3	Lemon
Red	3	Apple
Red	3	Apple

- The highlighted features are the exact same, except for the labels, so the output will be determined once the pipeline is written
- Family of Decision Tree Learning Algorithms:
  1. ID3
  2. C4.5
  3. C5.0
  4. CART
- Each give a procedure to “ask which questions, and when?”

- CART: Classification and Regression Trees:
  - A root node (the input at the top) receives the entire training set
  - Each node asks a true/false about one of the features
  - The data is split into two subsets based on the answer to the question
  - Two child nodes are created with the new subsets as input
  - The goal is to produce the “purest possible distribution of labels in each node”
  - Gini Impurity: A quantity measuring the amount of uncertainty at each node to pinpoint the effectiveness of asking each question
  - Information Gain: How much a question reduces uncertainty. Helps find a question that reduces uncertainty the most
  - When there are no new questions to ask, a leaf is added to the tree and that leaf is predicted as the correct label

Credit: Google Developers







- What types of questions can we ask?

        - A question can be thought of based on each feature listen in each row
        - Each becomes a new category for partitioning the data into subsets
    - Impurity
        - Chance of being incorrect if you randomly assign a label to an example in the same set
        - If no chance of being wrong, then impurity = 0

• Avg Impurity = ⅘ * Subset 1 + ⅕ * Subset 2 • Information Gain = 0.64 - 0.5 = 0.14

Building and printing the tree can be found in the code linked above.

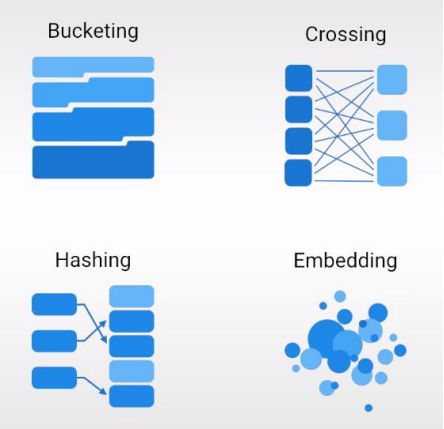
- **#9 - Intro to Feature Engineering with TensorFlow:**

Example: The Census dataset: • **50k examples** • **6 numeric features** • **9 categorical features** **Predict income based on attributes about a person**

    - Numeric Attributes: Sometimes the raw numeric value can be used as a feature, if the relationship is linear
    - Bucketing: Transforms a numeric feature into several categories. One categorical feature is created for each bucket.
    - Vocabulary: Possible outcomes for a feature
    - Categorical Features:
        1. Use the raw value only if there are only a small amount of possibilities
    - Feature Crossing: Create new features that are combinations of existing ones
    - Hashing: Represent a categorical feature with a large vocabulary
        1. 1 bit for each possible value in the vocabulary
        2. If the vocabulary is unknown, hashes are used to compute the bits
        3. Limits memory usage

4. Saves time

- Embeddings:
    1. Powerful way to represent large vocabularies
    2. Learned automatically
    3. Dense vectors vs. one-hot (sparse)
    4. Large vocabulary
    5. Concepts (vs specifics)
    6. Are dense with concepts
    7. Gives the classifier inside knowledge about what features mean
        - https://projector.tensorflow.org
          Visual Representation (source: Google)
        - https://goo.gl/Dfpb7W
        - https://goo.gl/2SxrYK



- Code to try out that uses TF methods to sort and improve features: https://goo.gl/K9dVqv

Unity 3D ML Agents:

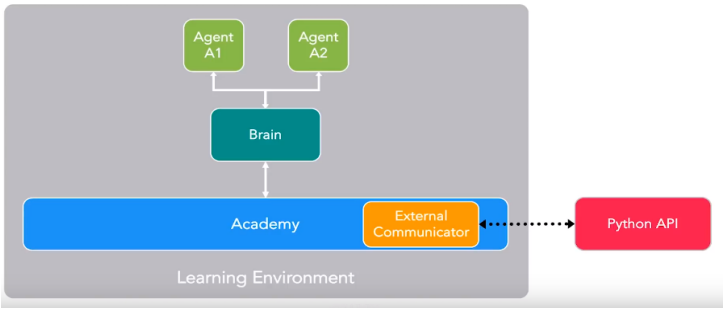
The following examples are programmed in C# and the Unity 3D Engine, a great developing engine for responsive 3D games:

- Prerequisites:
  - Unity 3D Software and standalone IDE
  - Unity ML Agents repository imported into project
  - Anaconda
  - TensorFlow Modules and prerequisites
  - NumPy Modules

- Machine Learning: Instead of hard coding computer interactions, allowing the computer to interact based on data (*learning* from data)
- ML Model: A way to take an image and map it into something else
- An example of using ML in games:
  - Take input of player controls
  - Output the best movement to take or what button to press
- Training Methods:
  - Reinforcement Learning:
    - Learn through rewards (primary learning signal through a scalar value)
    - Trial-and-error
    - Super-speed simulation
    - Agent becomes “optimal” at task
  - Imitation Learning: ← Best algorithm for adapting to user interaction
    - Learn through demonstrations
    - No rewards necessary
    - Real-time interaction
    - Agent becomes “human-like” at task

- Unity ML-Agents Workflow:

Create Environment (Unity) → Train Brains (Python/TF) → Embed Brains in Environment (Unity/TF)



Credit Image: Unity

- Academy: Allows Unity to communicate with outside languages and APIs
- Brain: Make decisions (Neural Networks)
- Agents: Observes the world and uses observations via the Brain to act
- There can be as many Brains and Agents as needed